NTNU
Norwegian University of
Science and Technology

Practical Anonymous Communication with Homomorphic Encryption

Angelo, Imre

Submission date: July 2026
Main supervisor: Associate Professor Park, Jeongeun, NTNU

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Many anonymous communication systems have been proposed and implemented with the goal of allowing users to exchange messages over a network without revealing who is communicating with whom. Most of these designs rely on the *threshold model* to provide anonymity, wherein some components of their infrastructure must be behaving honestly. Systems with stronger anonymity guarantees generally suffer from poor scalability or are impractical to instantiate.

This thesis examines a new construction called a (Somewhat) Practical Anonymous Router (sPAR), which addresses the trust and practicality concerns of existing systems. The construction relies only on well-established Fully Homomorphic Encryption (FHE) schemes, making it realistically implementable. While sPAR presents a promising new avenue for constructing anonymous communication systems, it remains a theoretical construction. A concrete implementation and a deeper investigation is therefore needed to evaluate its practical performance and viability in larger networks.

In an effort to investigate sPAR's viability, this thesis aims to implement the scheme and conduct a structured evaluation of its performance with respect to the number of participants in the system.

Research Questions:

- What is the computational cost of sPAR, and how does it compare to the theoretical complexity described in the original paper?
- How does the performance of sPAR scale with the number of participants, and at what point does it become impractical?

Approved: 2026-04-01 – Associate Professor Park, Jeongeun, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Sammendrag

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures

List of Tables

List of Algorithms

2.1 Fast Base Extension 8
3.1 Encrypt RGSW-BV 19
3.2 Encrypt RGSW (optimized) 20
3.3 Optimized Branchless Signed LSB Decomposition ($B = 2^b$) 21

Chapter 1
Background

Anonymous communication aims to enable users to exchange messages over a network without revealing who is communicating with whom, thereby protecting both sender and receiver identities. Many such systems have been proposed and implemented over the years, such as Tor, I2P, and mixnets. Most of these designs depend on the *threshold model* to provide anonymity, requiring at least some components of their infrastructure to behave honestly. In practice, this assumption is increasingly strained by large-scale surveillance, correlation attacks, and the growing centralization of internet infrastructure [SEV+15].

Systems with stronger anonymity guarantees based on Multi-Party Communication (MPC) exist. However, these suffer from poor scalability due to their high interactivity, making them impractical for large-scale networks. A Dining Cryptographers network (DC-net), for example, can provide anonymity even if there are no trusted components in the network infrastructure. Doing so requires every participant to interact with every other participant, for every round of messages sent.

Recently, a new line of research has proposed fundamentally different designs, aiming to offer strong anonymity even in the presence of compromised or adversarial components in the network actively attempting to break anonymity. The first such design was the Non-Interactive Anonymous Router (NIAR) [SW21]. This paper proposed using Multi-Client Functional Encryption (MCFE) to allow a single, untrusted router to shuffle or permute messages from a set of clients *obliviously*; without learning anything about the permutation. Thereby making it infeasible for the router to link any of the messages to its sender.

NIAR was a breakthrough in anonymous communication, not only showing the theoretical possibility of designing anonymous routers, but also that such routers can be *non-interactive*: clients only send one message to the router and do not need to perform any additional interaction to achieve anonymity. Using a centralized (anonymous) router in this way yields similar benefits to existing MPC-based systems, without needing any interactions between clients. However, there are a number of problems that makes NIAR impractical, if not impossible, to implement in practice.

1. The cryptographic components required by NIAR are extremely computationally expensive or not known to be implementable in practice.
2. Achieving anonymity for recipients assumes the existence of a very strong primitive called *indistinguishability obfuscation with sub-exponential security*. The practicality of such a construction is unknown.
3. Reusing the same setup across multiple rounds makes messages from the same sender linkable, unless the expensive setup phase is rerun for every round

- Anonymous communication
- Current solutions
- Problems with current solutions
- FHE as a new approach
- sPAR
- RSA is multiplicatively homomorphic, but (follow-up) paper theorized a *fully* homomorphic scheme may be possible [Gen09; RAD78]

Chapter 2

Related Works

This section details the fundamental building blocks of the sPAR protocol, and the protocol itself. To achieve a reasonable depth the protocol utilizes the *external product*, a primitive from TFHE bootstrapping that allows homomorphic multiplication of binary digits with near-constant noise bounds — enabling a large number of binary multiplication without consuming a level/bootstrapping. This primitive was ported to BGV-RNS for this thesis, which does not natively allow digit decomposition as normally used by the external product.

2.1 Fully Homomorphic Encryption

Homomorphic Encryption (HE) allows computation directly on encrypted data: an untrusted party can evaluate a function over ciphertexts without learning anything about the underlying plaintexts. Schemes are classified by the computations they support — Somewhat Homomorphic Encryption (SHE) and *leveled* schemes evaluate circuits of bounded (multiplicative) depth, while FHE supports circuits of arbitrary depth, typically by resetting the accumulated noise through bootstrapping.

Modern FHE schemes operate on polynomials in the ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ and base their security on the hardness of the (decision) Ring Learning With Errors (RLWE) problem: distinguishing samples of the form $(a, a \cdot s + \epsilon)$ from uniformly random pairs in R_q^2. This section focuses on the BGV scheme [BGV11] used by the implementation of this thesis; the required TFHE primitives are presented in a later section.

Definition 2.1. RLWE ciphertext $\overrightarrow{\mathbf{x}} \in R_q^2$ containing a uniformly random *public element* a and the masked element $b = a \cdot s + \Delta m + \epsilon$ derived from a persistent secret key s and randomly sampled noise ϵ, where Δ is the message scaling factor and $m \in R_t$ is the plaintext message being encrypted.

$$\mathsf{RLWE}(m) = (b, a) = (a \cdot s + \Delta m + \epsilon, a) \tag{2.1}$$

$$s \xleftarrow{\$} \{-1, 0, 1\}^N, \quad \epsilon \xleftarrow{\$} \chi, \quad a \xleftarrow{\$} R_q \tag{2.2}$$

An RLWE ciphertext $\overrightarrow{\mathbf{x}}$ encrypting a message m under the secret key s is denoted $\mathsf{RLWE}_s(m)$, but if the secret key is understood from context then $\mathsf{RLWE}(m)$ is typically used.

Most HE schemes support homomorphic addition and multiplication, and admit one of the two for *free*, in the sense that applying the operation directly to the ciphertexts applies it to the underlying plaintexts, without any additional machinery. For example, RSA has free multiplication, while schemes based on RLWE have free addition.

Definition 2.2. RLWE addition.

$$\overrightarrow{\mathbf{x}} + \overrightarrow{\mathbf{y}} = (b_x, a_x) + (b_y, a_y) = \mathsf{RLWE}_s(m_x + m_y) \tag{2.3}$$

Proof.

$$\begin{gathered} (a_x s + \Delta m_x + \epsilon_x, a_x) + (a_y s + \Delta m_y + \epsilon_y, a_y) \\ ((a_x + a_y)s + \Delta(m_x + m_y) + (\epsilon_x + \epsilon_y), a_x + a_y) \\ \text{Let } a = (a_x + a_y) \text{ and } \epsilon = (\epsilon_x + \epsilon_y) \\ \therefore \overrightarrow{\mathbf{x}} + \overrightarrow{\mathbf{y}} = (as + \Delta(m_x + m_y) + \epsilon, a) = \mathsf{RLWE}_s(m_x + m_y) \end{gathered}$$

□

Homomorphic multiplication in BGV is considerably more complicated and expensive: the product of two ciphertexts has three elements and multiplied noise terms, and requires *relinearization* — a form of key switching — to return to two elements and manage the noise growth without bootstrapping. As sPAR performs no such multiplications, the procedure is not detailed further.

Definition 2.3. Modulus Switching The process of converting a ciphertext $\overrightarrow{\mathbf{x}} \in R_Q^2$ encrypted under a ciphertext modulus Q into a ciphertext $\overrightarrow{\mathbf{x}}' \in R_{Q'}^2$ encrypted under a different modulus Q', while ensuring that both ciphertexts still decrypt to the same message.

Scaling the message by a factor of Q'/Q and rounding to the nearest integer achieves this,

This process is typically the primary tool for noise management in leveled schemes, as scaling down the modulus also scales down the noise, allowing further computation without bootstrapping.

Definition 2.4. Bootstrapping The process of homomorphically evaluating the decryption circuit to reset the noise of a ciphertext, allowing further computation. Bootstrapping is what turns a leveled scheme into a *fully* homomorphic one.

Definition 2.5. Keyswitching The process of converting a message to being encrypted under a different secret key.

2.1.1 BGV

The implementation of this thesis is in the BGV cryptosystem, which places the message in the *least* significant bits of the ciphertext: $\Delta = 1$ and the noise is scaled by the plaintext modulus, $\epsilon = t \cdot \epsilon_{\mathsf{rlwe}}$, giving $b = a \cdot s + m + t\epsilon_{\mathsf{rlwe}}$. Decryption computes $m = [[b - a \cdot s]_q]_t$, which is correct as long as the noise does not wrap around q.

2.2 Residue Number Systems

In BGV, BFV and CKKS the ciphertext modulus Q is large — much larger than a computer word (typically 64 bits on modern computers). Multi-precision arithmetic is known to be slow due to its serial nature, so implementations avoid it by using a Residue Number System (RNS), which represents large numbers as a set of smaller, independent residues. RNS variants of BGV, BFV and CKKS are the standard in practice; OpenFHE, for example, only supports the RNS versions of these schemes.

Definition 2.6. Canonical Form We are interested in polynomials in the ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$ with degree (up to) $N - 1$ and coefficients in $[-Q/2, Q/2)$ (centered modulo representation), and different ways to represent this polynomial. The canonical form (**??**) of such a polynomial a is the form everyone is familiar with.

$$a = c_0 + c_1 X + \cdots + c_{N-1} X^{N-1} = \sum_{i=0}^{N-1} c_i X^i \tag{2.4}$$

Definition 2.7. Coefficient Form As polynomials have a well-known/predictable structure, we can omit the redundant X variable and represent $a \in R_Q$ uniquely by its coefficients. I will refer to (only) this representation (**??**) as the **coefficient** form.

$$a = (c_0, c_1, \ldots, c_{N-1}) \tag{2.5}$$

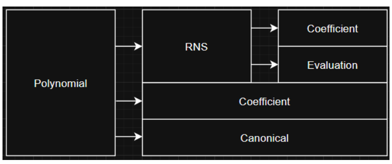
Let $Q = q_0 \ldots q_{k-1}$ with $\gcd(q_i, q_j) = 1 \ \forall\, i \neq j$, so that the Chinese Remainder Theorem (CRT) allows decomposition into smaller residues. If $\max_i\{q_i\}$ fits in a computer word then we can use CRT on the coefficients of a polynomial to work exclusively with integers that fit inside a computer word.

Definition 2.8. CRT Form Given a polynomial $a \in R_Q$ where $Q = q_0 q_1 \ldots q_{k-1}$ is chosen as a product of k small primes (small meaning they each fit inside a computer register), each coefficient c_i is uniquely represented by its k residues via the Chinese Remainder Theorem. Then the **CRT form** of a (**??**) is a collection of smaller polynomials $(\bmod q_i)$ called *towers*, *residues* or *limbs*.

$$\begin{pmatrix} a \mod q_0 \\ a \mod q_1 \\ \vdots \\ a \mod q_{k-1} \end{pmatrix} = \begin{pmatrix} c_0 \bmod q_0 & c_1 \bmod q_0 & \ldots & c_{N-1} \bmod q_0 \\ c_0 \bmod q_1 & c_1 \bmod q_1 & \ldots & c_{N-1} \bmod q_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod q_{k-1} & c_1 \bmod q_{k-1} & \ldots & c_{N-1} \bmod q_{k-1} \end{pmatrix} \tag{2.6}$$

Addition and subtraction of two polynomials in R_Q is equivalent to element-wise addition/subtraction of their CRT forms (modulo q_i). As each element fits within a computer word, no carries propagate between them, and all pairs of elements can be added or subtracted in parallel.

Remark 2.9. Rather confusingly, OpenFHE calls this form *Coefficient* format. Both RNS forms are represented by an `DCRTPoly` object, so a polynomial in CRT form is represented by a `DCRTPoly` object in `Format::Coefficient` mode.



2.2.1 Number Theoretic Transform

The Number Theoretic Transform (NTT) converts a negacyclic convolution in the coefficient domain into point-wise multiplication in the NTT domain — and multiplication in R_{q_i} is exactly such a convolution of the coefficient vectors. Applying the NTT to each residue (row) of the CRT form (**??**) therefore makes polynomial multiplication element-wise as well. The negacyclic NTT of size N requires each modulus to satisfy $q_i \equiv 1 \pmod{2N}$, which constrains the choice of RNS primes.

Definition 2.10. Double CRT/NTT/Evaluation Form By applying the NTT to the CRT form (**??**) we gain the ability to perform polynomial multiplication in parallel, without losing the ability to perform addition/subtraction in parallel.

$$\begin{pmatrix} \mathsf{NTT}(a \bmod q_0) \\ \mathsf{NTT}(a \bmod q_1) \\ \vdots \\ \mathsf{NTT}(a \bmod q_{k-1}) \end{pmatrix} = \begin{pmatrix} \hat{a}_{0,0} & \hat{a}_{0,1} & \cdots & \hat{a}_{0,N-1} \\ \hat{a}_{1,0} & \hat{a}_{1,1} & \cdots & \hat{a}_{1,N-1} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{a}_{k-1,0} & \hat{a}_{k-1,1} & \cdots & \hat{a}_{k-1,N-1} \end{pmatrix} \tag{2.7}$$

The evaluation form (**??**) is the preferred form, as it supports parallel addition, subtraction and multiplication, but it still does not natively support *all* necessary operations. Because switching between canonical/vector, CRT and NTT mode is relatively expensive — requiring (inverse) CRTs and NTTs — much of the relevant literature [GHS12; BEHZ16; HPS18; KPZ21] improves upon existing functions by finding new approaches and approximations that limit the number of format-switches required.

$$\text{Coefficient} \underset{\text{iCRT}}{\overset{\text{CRT}}{\rightleftarrows}} \text{CRT} \underset{\text{iNTT}}{\overset{\text{NTT}}{\rightleftarrows}} \text{Evaluation (DCRT)}$$

2.2.2 RNS Base Conversion

Several ciphertext maintenance operations — most importantly the key-switching procedures of the next section — temporarily extend the ciphertext modulus Q by an auxiliary modulus P and later scale back down. In RNS this corresponds to moving a polynomial between different RNS bases. I start by introducing some terminology.

Definition 2.11. RNS Basis. Given a large modulus $Q = \prod_{i=0}^{k-1} q_i$ composed of pairwise coprime factors, we define the *RNS basis* $\mathcal{B}_Q = \{q_0, q_1, \dots, q_{k-1}\}$ as the set of RNS moduli used in the RNS/CRT representation (**??**).

Definition 2.12. Base conversion Let $\mathcal{B}_Q = \{q_0, \dots, q_{k-1}\}$ and $\mathcal{B} = \{b_0, \dots, b_{\ell-1}\}$ be RNS bases with $\gcd(Q, B) = 1$, where $B = \prod_j b_j$. Base conversion is the map that takes the representation of x in $\mathcal{B}_Q$ to the representation of $[x]_Q$ in $\mathcal{B}$, i.e.

$$\big([x]_{q_0}, \dots, [x]_{q_{k-1}}\big) \longmapsto \big([x]_{b_0}, \dots, [x]_{b_{\ell-1}}\big).$$

When the residues in $\mathcal{B}$ are appended to those in $\mathcal{B}_Q$, so that x becomes represented in the *extended* basis $\mathcal{B}_Q \cup \mathcal{B}$ for the modulus QB, the operation is also referred to as **base extension**.

Remark 2.13. Extend mod notation for RNS polynomials: $[a]_{\mathcal{B}_Q} = ([a]_{q_0}, \dots, [a]_{q_{k-1}})$ for the basis $\mathcal{B}_Q = \{q_i\}_{i=0}^{k-1}$ means a must be in RNS form (TODO: distinguish CRT vs NTT mode... maybe $[a]_{\mathcal{B}_Q}^{\mathsf{NTT}}$?)

The naive approach to base conversion reconstructs a from its residues via the CRT and then reduces the result modulo each modulus of the new basis. This reintroduces the multi-precision arithmetic that RNS was meant to avoid. A cheap but approximate alternative exists, described next.

Fast Base Conversion

Fast base conversion, introduced by [BEHZ16], performs the conversion directly on the residues (in CRT mode), avoiding the CRT reconstruction and multi-precision integers entirely. The price is approximation: (**??**) produces not x but $x + \alpha_x Q$ for some integer $\alpha_x \in [0, k)$, as computing the exact overflow α_x would require costly operations in RNS.

$$\mathsf{FastBConv}(x, \mathcal{B}_Q, \mathcal{B}) = \left(\left[\sum_{i=0}^{k-1} \left[x_i \left(\frac{Q}{q_i} \right)^{-1} \right]_{q_i} \times \frac{Q}{q_i} \right]_b \right)_{b \in \mathcal{B}} \tag{2.8}$$

Fast Base Extension

We can use (**??**) as a subroutine in base extension: only the new residues $\mathcal{B}_Q \longmapsto \mathcal{B}_P$ need to be calculated, and since $\mathcal{B}_{QP} = \mathcal{B}_Q \cup \mathcal{B}_P$ the original residues are simply kept.

Algorithm 2.1 Fast Base Extension

Input: $[a]_{\mathcal{B}_Q}$ and a basis $\mathcal{B}_P$
Output: $[a + \epsilon]_{\mathcal{B}_{QP}}$, where $\epsilon = \alpha_a Q$ for some $\alpha_a \in [0, k)$
1: $a_Q := \mathsf{InverseNTT}(a)$ ▷ Convert to CRT form
2: $a_P := \mathsf{FastBConv}(a_Q, \mathcal{B}_Q, \mathcal{B}_P)$
3: **return** $a \cup \mathsf{NTT}(a_P)$

Notably, (**??**) is approximate as a *conversion* but exact as an *extension*: the extended residue vector over $\mathcal{B}_Q \cup \mathcal{B}_P$ is an exact RNS representation of the lifted integer $x' = [x]_Q + \alpha_x Q \in \mathbb{Z}_{QP}$, since $\alpha_x Q \equiv 0 \pmod{q_i}$ keeps the original (reused) Q-residues consistent with x'. The RNS schemes are designed to tolerate this overflow, which is removed — or shrunk to the small α_x — by a subsequent division during mod down.

Remark 2.14. The factors $[(\frac{Q}{q_i})^{-1}]_{q_i}$ and $\frac{Q}{q_i}$ can be pre-calculated and stored in memory (will be discussed in **??**).

Modulus Switching

In RNS, modulus switching — also referred to as *mod down* — from Q to $Q' = Q/q_{k-1}$ amounts to *dropping the last limb* with a small correction, computed limb-wise without

CRT reconstruction:

$$[c']_{q_i} = \left[q_{k-1}^{-1}\,(c-\delta)\right]_{q_i}, \quad 0 \le i < k-1, \tag{2.9}$$

where $\delta \equiv c \pmod{q_{k-1}}$ and, for BGV, $\delta \equiv 0 \pmod{t}$ so that decryption correctness is maintained. The noise shrinks by a factor q_{k-1}, matching the classical modulus switch.

Approximate Mod Down $QP \to Q$

The reverse operation, switching from the extended modulus QP back down to Q, uses (**??**) to convert the P-part back to $\mathcal{B}_Q$ before dividing by P [GHS12; HPS18]:

$$\mathsf{ModDown}(x) = \left[P^{-1}\,(x - \mathsf{FastBConv}([x]_{\mathcal{B}_P}, \mathcal{B}_P, \mathcal{B}_Q))\right]_{\mathcal{B}_Q} \tag{2.10}$$

For BGV the converted term must additionally be corrected modulo t to preserve the plaintext [KPZ21]. This operation also corrects the fast base conversion error: the conversion overflow αP is divided by P, shrinking it to the small additive term α — which is why the approximation in (**??**) is tolerable.

2.3 Noise Management Techniques in RNS

A problem that arises in several areas of FHE is that of multiplying a ciphertext by some large, known factor $X \in R_Q$ while keeping the error low. Performed naively, the noise scales together with the message:

$$(a \cdot s + \Delta m + \epsilon) \cdot X = aX \cdot s + \Delta X m + X\epsilon. \tag{2.11}$$

Several core operations require computing the product of a ciphertext element with noisy auxiliary material; performed naively, this scales the noise by a factor as large as q. This section covers the standard techniques for controlling this growth: decomposition (BV), modulus extension (GHS), and their combination (Hybrid).

These techniques typically occur in the literature in the context of key switching, but we remove them entirely from that context and consider only the techniques themselves.

2.3.1 Brakerski-Vaikuntanathan

Brakerski and Vaikuntanathan [BV11] first proposed decomposing ciphertext elements into small digits as a means of controlling the noise growth of this operation. The idea has since been generalized into the idea of *gadget decomposition* and applied in a variety of forms, including digit and CRT (RNS) decomposition. Following the convention of [KPZ21; BAB+22], we refer to this family of decomposition-based techniques collectively as *BV*.

Definition 2.15. Digit Decomposition takes an integer $\gamma \in \mathbb{Z}_q$ (for a modulus $q \geq 2$), base β and *decomposition parameter* ℓ, and maps γ to its ℓ most significant *signed* base-β digits $\gamma_i \in [-\beta/2, \beta/2)$:

$$\gamma = \sum_{i=0}^{\ell-1} \gamma_i \frac{q}{\beta^{i+1}} + \epsilon, \qquad |\epsilon| \leq \frac{q}{2\beta^\ell}, \tag{2.12}$$

where the remaining, least significant digits are simply dropped; keeping all digits ($\beta^\ell \geq q$) makes the decomposition exact ($\epsilon = 0$). The *decomposition of* γ is the ordered list of the ℓ digits and is denoted $\mathsf{Decomp}(\gamma) = (\gamma_0, \gamma_1, \ldots, \gamma_{\ell-1})$.

Remark 2.16. When q is not a power of β (e.g. the prime moduli of **??**) the entries q/β^{i+1} are rounded to the nearest integer; the rounding error is absorbed into ϵ.

By construction, digit decomposition satisfies $\langle \mathsf{Decomp}(\gamma), g\rangle = \gamma$ for the fixed vector $g = (q/\beta, \ldots, q/\beta^\ell)$. The particular form of g is unimportant; what matters is that the identity holds while the entries of $\mathsf{Decomp}(\gamma)$ remain small, since then, scaling g by any $x \in \mathbb{Z}_q$,

$$\langle \mathsf{Decomp}(\gamma), x \cdot g\rangle = x \cdot \gamma \pmod q,$$

the product $x \cdot \gamma$ is computed with γ entering only through its small digits. The gadget property abstracts exactly this: it requires nothing of the maps $D(\gamma) = \mathsf{Decomp}(\gamma)$ and $P(x) = x \cdot g$ beyond that their inner product (approximately) recovers $x \cdot \gamma$ and that the entries of $D(\gamma)$ are small.

Definition 2.17. Gadget Property. Two mappings $P(x), D(y) \in R_q^\ell$ are said to have the gadget property with *quality* (digit bound) β and *precision* ϵ iff. they satisfy (**??**) for any pair of integers $(x, y) \in \mathbb{Z}_q^2$.

$$\langle P(x), D(y)\rangle = x \cdot (y + \epsilon_y) \pmod q, \qquad |\epsilon_y| \leq \epsilon, \quad \|D(y)\|_\infty \leq \beta \tag{2.13}$$

If $\epsilon = 0$ then $P(x), D(y)$ are said to have the *exact* gadget property.

Definition 2.18. Gadget Decomposition generalizes the notion of digit decomposition by replacing the base β with a *gadget vector* $g = (g_0, g_1, \ldots, g_{\ell-1})$ containing ℓ elements, such that $D(\gamma) = \mathsf{Decomp}^g(\gamma)$ and $P(x) = x \cdot g$ satisfy Def. **??**.

$$\gamma + \epsilon_\gamma = \sum_{i=0}^{\ell-1} \gamma_i g_i \quad \Longleftrightarrow \quad \langle \mathsf{Decomp}^g(\gamma), g\rangle = \gamma + \epsilon_\gamma$$

All of the above extends coefficient-wise to polynomials $\gamma \in R_q$: the digits become polynomials with small coefficients, and the bounds β and ϵ apply to $\|\cdot\|_\infty$.

RNS Instantiation

Digit decomposition is not directly compatible with RNS: the digits of γ are defined by its canonical value, which is not available limb-wise. Extracting them requires reconstructing the coefficients — an inverse NTT followed by an inverse CRT — reintroducing the multi-precision detour that RNS is meant to avoid (**??**).

The CRT structure itself, however, is a gadget. The first RNS instantiation of BV [BEHZ16] decomposes into residues:

$$D_{\mathsf{CRT}}(\gamma) = \left([\gamma]_{q_0}, \ldots, [\gamma]_{q_{k-1}}\right), \qquad P_{\mathsf{CRT}}(x) = (x \cdot e_0, \ldots, x \cdot e_{k-1}), \qquad e_j = \left[\left(\frac{Q}{q_j}\right)^{-1}\right]_{q_j} \cdot \frac{Q}{q_j}, \tag{2.14}$$

where the e_j are the CRT idempotents: $e_j \equiv 1 \pmod{q_j}$ and $e_j \equiv 0 \pmod{q_{j'}}$ for $j' \neq j$, so that $\sum_j [\gamma]_{q_j} e_j \equiv \gamma \pmod{Q}$ — the exact gadget property with $\ell = k$. Both maps are computed natively in RNS: D_{CRT} simply reads off the limbs, and P_{CRT} scales them. The drawback is the digit bound $\beta = \max_j q_j/2$, a full machine word, which enters the noise bounds of **??** linearly. [HPS18] (HPS) further optimized this construction for key switching using precomputed factors, as remarked in **??**.

To shrink the digits, the CRT gadget is composed with digit decomposition applied *per residue*: each limb $[\gamma]_{q_j}$ is an ordinary word-sized integer, so it can be decomposed into ℓ signed base-β digits limb-wise, without reconstruction. The composed gadget

$$D_{\mathsf{BV}}(\gamma) = \left(\mathsf{Decomp}\left([\gamma]_{q_j}\right)\right)_{j=0}^{k-1}, \qquad P_{\mathsf{BV}}(x) = \left(x \cdot \beta^i e_j\right)_{j,i} \tag{2.15}$$

has $k\ell$ digits bounded by $\beta/2$, and is exact whenever $\beta^\ell \geq \max_j q_j$. This is the closest direct translation of TFHE-style decomposition to RNS, and one of the two variants implemented in this thesis (**??**).

2.3.2 Gentry-Halevi-Shoup

The GHS method [GHS12] controls noise without any decomposition ($\ell = 1$). Instead, the modulus is temporarily extended from Q to QP for an auxiliary modulus P (**??**), and the auxiliary material is pre-scaled by P:

$$D_{\mathsf{GHS}}(\gamma) = \mathsf{ModUp}(\gamma) \in R_{QP}, \qquad P_{\mathsf{GHS}}(x) = [P \cdot x]_{QP}, \tag{2.16}$$

where ModUp is the fast base extension of Algorithm **??**. The product $D_{\mathsf{GHS}}(\gamma) \cdot P_{\mathsf{GHS}}(x) = P \cdot x\gamma \pmod{QP}$ is brought back to Q with $\mathsf{ModDown}$ (**??**), dividing by P. The single "digit" γ remains as large as Q, but any noise attached to the pre-scaled material is divided by $P \approx Q$ on the way down, shrinking to insignificance; the price is a small additive error $\epsilon > 0$ (the FastBConv overflow and the ModDown rounding) and auxiliary material living in the larger ring R_{QP}. GHS is well established for key switching; its use in the external product (**??**) is not, and is developed in this thesis (**??**).

2.3.3 Hybrid

BV and GHS are endpoints of a spectrum. Hybrid key switching [KPZ21] groups the k limbs into dnum digits of $\alpha = \lceil k/\mathsf{dnum} \rceil$ limbs each — a coarser CRT gadget — and applies the GHS modulus extension to control the remaining per-digit noise, requiring only $P \gtrsim Q^{1/\mathsf{dnum}}$. Setting $\mathsf{dnum} = k$ (no extension needed) recovers BV, while $\mathsf{dnum} = 1$ recovers GHS; the "hybrid" implementation of this thesis uses $\mathsf{dnum} = 1$ and is therefore effectively GHS.

2.4 TFHE Bootstrapping Primitives

TFHE [CGGI16; CGGI18] is optimized for computations of large multiplicative depth on small (binary) plaintexts, and is best known for its fast bootstrapping procedure. This section covers only the machinery that bootstrapping — and this thesis — is built from: RGSW ciphertexts and the external and internal products. Bootstrapping itself (blind rotation, sample extraction, LWE key switching) is beyond the scope of this work.

GSW encryption was introduced over plain LWE by Gentry, Sahai and Waters [GSW13]. The ring variant (RGSW) appears in FHEW [DM14], which uses RGSW accumulators for bootstrapping, and in SHIELD [KGV14] around the same time. The external product — the asymmetric RGSW $\times$ RLWE multiplication, as opposed to GSW's original symmetric product — is due to Chillotti, Gama, Georgieva and Izabachène [CGGI16] (journal version [CGGI18]), with a concurrent equivalent (the "hybrid product") by Brakerski and Perlman [BP16]. The internal product (RGSW $\times$ RGSW) is the ring version of GSW's original multiplication.

Remark 2.19. The torus pivot, once and for all TFHE is natively formulated over the (discretized) torus, with TLWE/TGSW in place of RLWE/RGSW. The discretized torus $q^{-1}\mathbb{Z}/\mathbb{Z}$ is isomorphic to $\mathbb{Z}_q$ via $x \mapsto qx$ [Joy21], under which the torus gadget entries $1/B^j$ become q/B^j. All statements transfer verbatim; we use ring notation over R_Q throughout, with RLWE ciphertexts as in **??** and gadget pairs (D, P) satisfying the (approximate) gadget property of Def. **??**.

2.4.1 RGSW Ciphertexts

Fix the radix gadget of Def. **??**, $g = P(1) = (q/B, \ldots, q/B^\ell)^T$, and the corresponding *gadget matrix*

$$G = I_2 \otimes g \in R_Q^{2\ell \times 2}. \tag{2.17}$$

Definition 2.20. RGSW Ciphertext An RGSW encryption of $\mu \in R$ is

$$C = Z + \mu \cdot G \in R_Q^{2\ell \times 2}, \qquad \text{each row of } Z \text{ an RLWE encryption of } 0. \tag{2.18}$$

Structurally, an RGSW ciphertext is 2ℓ RLWE ciphertexts encrypting gadget-scaled copies of μ: the rows of one block encrypt $\mu \cdot g_j$ directly, and the rows of the other encrypt $-s \cdot \mu g_j$. In particular, RGSW is additively homomorphic (matrix addition), and μ can be recovered by decrypting any row of the $\mu \cdot g_j$ block.

2.4.2 External Product

Gadget decomposition extends to an RLWE ciphertext $c = (c_0, c_1)$ by decomposing each component:

$$G^{-1}(c) = \big(D(c_0) \,\big|\, D(c_1)\big) \in R^{2\ell}, \qquad \text{satisfying } G^{-1}(c) \cdot G = (c_0, c_1) + (e_0, e_1), \quad \|e_i\|_\infty \le \varepsilon. \tag{2.19}$$

Definition 2.21. External Product The external product $\boxdot$ of an RGSW ciphertext C and an RLWE ciphertext c is [CGGI16; BP16]

$$C \boxdot c = G^{-1}(c) \cdot C \in R_Q^2. \tag{2.20}$$

Correctness follows in one line by expanding (**??**):

$$G^{-1}(c) \cdot (Z + \mu_C G) = \underbrace{G^{-1}(c) \cdot Z}_{\text{decomposition–noise term}} + \mu_C \cdot \big(c + e_{\text{dec}}\big) = \mathsf{RLWE}(\mu_C \cdot \mu_c) + \text{noise},$$

i.e. the result encrypts the product of the two messages, with worst-case noise

$$\|v_\boxdot\|_\infty \lesssim \underbrace{2\ell\, N\, \beta\, B_Z}_{\langle D(c_i),\, \vec{e}_Z \rangle} + \underbrace{\|\mu_C\|_1 \cdot \varepsilon}_{\mu_C \cdot e_{\text{dec}}} + \|\mu_C\|_1 \cdot \|v_c\|_\infty, \tag{2.21}$$

where β is the digit bound of D, ε its precision, B_Z the RGSW noise and v_c the noise of the RLWE input (statement only; proof deferred to Thm. **??**).

Two consequences are worth flagging. First, the noise is *asymmetric*: the RLWE input's noise enters scaled by μ_C, while the RGSW noise enters only through small digits. The RGSW message must therefore be small — typically $\mu_C \in \{0, 1\}$ or a monomial X^k — while μ_c is unconstrained. With $\|\mu_C\|_1 = 1$ the input noise passes through *unchanged*, so a chain of n external products grows the noise only linearly in n; this near-constant growth per multiplication is what TFHE bootstrapping — and the sPAR protocol — are built on. Second, the cost is 2ℓ ring multiplications plus the cost of the decomposition D.

2.4.3 Internal Product

The internal product $\boxtimes$ (RGSW $\times$ RGSW $\to$ RGSW) applies the external product row-wise, each row of C_1 being an RLWE ciphertext:

$$C_1 \boxtimes C_2 = \big(C_2 \boxdot \mathrm{row}_r(C_1)\big)_{r=1}^{2\ell}. \tag{2.22}$$

It costs 2ℓ external products, with the noise of (**??**) per row. Its role is composing RGSW ciphertexts (e.g. products of control bits); it is not used on the critical path of this thesis and is included for completeness.

2.4.4 Generalization

Nothing above used the radix structure of g — only the gadget property of (D, P). Any pair from **??** therefore yields a valid RGSW ciphertext and external product.

Theorem 2.22. *Let $D(a), P(b) \in R_Q^\ell$ satisfy the approximate gadget property (Def. **??**) with digit bound β and decomposition error ε. Then*

$$G = I_2 \otimes P(1) \tag{2.23}$$

$$G^{-1}(x) = \big(D(x_0) \,\big|\, D(x_1)\big) \tag{2.24}$$

define a valid RGSW ciphertext (**??**) *and external product* (**??**) *with* $\mathrm{RGSW}(\mu_C) \boxdot \mathrm{RLWE}(\mu_c) = \mathrm{RLWE}(\mu_C \cdot \mu_c)$*, provided the resulting noise remains below the decryption bound.*

Instantiating Thm. **??** with the gadgets of **??** gives external products computable natively in RNS: the BV-type gadgets are exact ($\varepsilon = 0$), while the extension-based gadget (GHS/Hybrid, **??**) is approximate, with $\varepsilon > 0$ from the FastBConv overflow and the ApproxModDown rounding, and the P-scaling absorbed into $P(\cdot)$.

Finally, a genuine (not merely notational) difference from the TFHE setting: in BGV the zero-encryptions carry the plaintext factor t — Z-rows of the form $(as + te, a)$ — so the noise bounds above acquire factors of t. This is treated in **??**.

2.5 (Somewhat) Practical Anonymous Router

–

Chapter 3
Methodology

3.1 Code/Library

– A result of this thesis: a C++ library which extends OpenFHE's BGV implementation to support RGSW operations
– Another result: sPAR (which was built on top of this library)
– The code base is split into three sections found in the `lib` directory:

1. `core` extends OpenFHE's `CryptoContext<T>` with the RGSW-like operations:
   - `EncryptRGSW` outputs the RGSW encryption of the input plaintext
   - `EvalExternalProduct` takes an RLWE and RGSW ciphertext
   - `EvalInternalProduct` takes two RGSW ciphertexts

2. `server` builds the sPAR server functions on top of `core`

3. `client` builds the sPAR client functions on top of `core`

– There are also two auxiliary directories `benchmarks` and `tests` that contain code to benchmark and unit test the functions, respectively.

3.2 Implementation Details (RGSW/Library)

– Provides an abstract interface called `ExtendedCryptoContext` which exposes RGSW functions (EncryptRGSW, EvalExternalProduct, EvalInternalProduct etc.)
– Two concrete implementations of the `ExtendedCryptoContext` interface based on:
  - the BV (double decomposition) gadget
  - the Hybrid approach (GHS in practice)

3.2.1 BV (Double Decomposition)

Power of Base

- As remarked in **??**, the HPS gadget is equivalent to limb look-up on RNS polynomials: $P_{\mathsf{HPS}}(a) \cong a \cdot I_k$
- **??** also shows we can use the standard BV gadget on each limb of an RNS polynomial.
- The resulting gadget contains mostly 0's, and the structure is known.

$$P_{\mathsf{HPS}}(a) \cong \begin{bmatrix} [a]_{q_0} & 0 & \cdots & 0 \\ 0 & [a]_{q_1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & [a]_{q_k} \end{bmatrix} = a \cdot I_k$$

compact storage $\swarrow$

$$\begin{bmatrix} [a]_{q_0} \\ [a]_{q_1} \\ \vdots \\ [a]_{q_k} \end{bmatrix} = \begin{bmatrix} c_0 \bmod p_0 & c_1 \bmod p_0 & \ldots & c_{N-1} \bmod p_0 \\ c_0 \bmod p_1 & c_1 \bmod p_1 & \ldots & c_{N-1} \bmod p_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod p_k & c_1 \bmod p_k & \ldots & c_{N-1} \bmod p_k \end{bmatrix}$$

decompose $\downarrow$

$$P_{\mathsf{BV}}(a) \cong \begin{bmatrix} [\hat{a}]_{q_0} \\ [\hat{a}]_{q_1} \\ \vdots \\ [\hat{a}]_{q_k} \end{bmatrix} = \begin{bmatrix} [a]_{q_0} & [aB]_{q_0} & \cdots & [aB^{\ell-1}]_{q_0} \\ [a]_{q_1} & [aB]_{q_1} & \cdots & [aB^{\ell-1}]_{q_1} \\ \vdots & \vdots & \ddots & \vdots \\ [a]_{q_k} & [aB]_{q_k} & \cdots & [aB^{\ell-1}]_{q_k} \end{bmatrix}$$

equivalent $\updownarrow$

$$\begin{bmatrix} [a \cdot g]_{q_0} \\ [a \cdot g]_{q_1} \\ \vdots \\ [a \cdot g]_{q_k} \end{bmatrix} \cong \begin{bmatrix} [a]_{q_0} & [aB]_{q_0} & \cdots & [aB^{\ell-1}]_{q_0} \\ [a]_{q_1} & [aB]_{q_1} & \cdots & [aB^{\ell-1}]_{q_1} \\ \vdots & \vdots & \ddots & \vdots \\ [a]_{q_k} & [aB]_{q_k} & \cdots & [aB^{\ell-1}]_{q_k} \end{bmatrix}$$

$\swarrow$ NTT

$$\left(a, aB, \ldots aB^{\ell-1}\right)_{\mathsf{DCRT}}$$

Note that $\hat{a}$ is a vector of polynomials (the gadget decomposition of a), a is a polynomial, and c_i are the coefficients of a. When we apply the second decomposition,

$a \to \hat{a}$, we get a 3-dimensional matrix of residues $\iff$ a 2-dimensional matrix of polynomial residues $\iff$ a vector of RNS polynomials $\therefore P_{\mathsf{BV}}(a) \leftrightarrow (a, aB, \ldots aB^{\ell-1})$.

Then constructing $P_Q(a)$ is in some way equivalent to constructing $\hat{a}$, and the only difference between $P_Q(a)$ and $\hat{a}$ in practice are the indices of the elements as we ignore the 0-towers. In other words we can construct $\hat{a}$ and in the external product skip any computations that involves a 0-tower.

RGSW-BV Data Structure

Each RGSW ciphertext is stored as a vector `std::vector<DCRTPoly>` of $2 \cdot k \cdot \ell$ RLWE ciphertexts (`DCRTPoly`), where k is the number of moduli in the RNS basis $\mathcal{B}_Q$ and ℓ is the decomposition parameter.

$$C = Z + \begin{pmatrix} [\mu]_{q_0} & [\mu]_{q_0}\beta & \cdots & [\mu]_{q_0}\beta^{\ell-1} & [\mu]_{q_1} & [\mu]_{q_1}\beta & \cdots & [\mu]_{q_1}\beta^{\ell-1} & 0 \\ 0 & 0 & \cdots & 0 & 0 & & & & \end{pmatrix}^{\mathsf{T}} \tag{3.1}$$

Definition 3.1. RGSW-BV Data Structure We define a data structure representing an RGSW ciphertext as a vector of RLWE ciphertexts. Let $G = I_2 \otimes P_{\mathsf{BV}}(1) = I_2 \otimes P_{\mathsf{HPS}}(g)$ and $G^{-1}(x) = [D_{\mathsf{HPS}}(x_0)|D_{\mathsf{HPS}}(x_1)]$ using the BV gadgets...

$$C = Z + mG = Z + m \begin{pmatrix} P(1) & 0 \\ \vdots & \vdots \\ P(1) & 0 \\ 0 & P(1) \\ \vdots & \vdots \\ 0 & P(1) \end{pmatrix} \tag{3.2}$$

Let $m_0 = m_0 \cdot P(1)$

Algorithm 3.1 Encrypt RGSW-BV

Input: Ring element m in NTT form and decomposition parameter ℓ
Output: RGSW as matrix of $2 \times k\ell$ polynomials

Definition 3.2. RGSW-BV External Product To calculate the external product using the BV decomposition using the RNS basis $\mathcal{B}_Q$ and decomposition parameter ℓ, we

And the external product:

$$x \boxdot Y = G^{-1}(x) \cdot Y \tag{3.3}$$
$$G^{-1}(x)(Z + m_y \cdot G) \tag{3.4}$$
$$G^{-1}(x) \cdot Z + m_y \cdot (G \cdot G^{-1}(x)) \tag{3.5}$$
$$G^{-1}(x) \cdot Z + m_y \cdot x \tag{3.6}$$

For the most optimal result, we flatten the loop to allow effective parallelization of each tower:

Algorithm 3.2 Encrypt RGSW (optimized)

Input: Polynomial m and param ℓ
Output: RGSW as matrix of $2 \times k\ell$ polynomials

```
Initialize a vector C of 2 · k · ℓ RLWE ciphertexts
for row ∈ [0, 2 · k · ℓ) in parallel do
    h ← ⌊row/(kℓ)⌋
    j ← ⌊(row mod kℓ)/ℓ⌋
    i ← ⌊(row mod kℓ) mod ℓ⌋
    z ← RLWE(0) = (c_0, c_1)
    c_h[j] ← [c_h[j] + mB^i]_{q_j}
    C[hkℓ + jℓ + i] = z
end for
return C
```

Decomposition

Digit extraction is performed in the coefficient domain. This is the main inefficiency in the HPS/BV-RNS gadget scheme. Let $\check{a} = ([d_0]_B, [d_1]_B, \cdots, [d_{k-1}]_B)$ denote the decomposition of a using ??. As $D_Q(a) = \left([a]_{q_0}, [a]_{q_1}, \cdots, [a]_{q_i}\right) = a$, the decomposition into $D_Q(\check{a})$ is equivalent to finding $(D_Q(d_0), D_Q(d_1), \cdots, D_Q(d_{k-1}))$.

	Digit 0 (B^0)	**Digit 1** (B^1)	$\cdots$	**Digit** $\ell - 1$ ($B^{\ell-1}$)
Tower q_0	$d_{0,0}$	$d_{0,1}$	$\cdots$	$d_{0,\ell-1}$
Tower q_1	$d_{1,0}$	$d_{1,1}$	$\cdots$	$d_{1,\ell-1}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
Tower q_{k-1}	$d_{k-1,0}$	$d_{k-1,1}$	$\cdots$	$d_{k-1,\ell-1}$

Table 3.1: The $k \times \ell$ structure of the Double-CRT Decomposition $D_Q(c)$.

Modulo Powers-of-2

The modulo operation is slow because it cannot be done in parallell. If we assume $B = 2^b$ is a power of 2 — which we can enforce by accepting $\log(B) = b$ as the input parameter instead of B — then we can use the fact that right-shift $(a \gg b) = a \bmod 2^b$ or something to make the modulo operation into a bit-wise (and thus easily parallellizable) operation.

Algorithm 3.3 Optimized Branchless Signed LSB Decomposition ($B = 2^b$)

Input: Coefficient $a \in \mathbb{Z}$, bit-width b, number of digits ℓ
Output: Vector of signed digits $\mathbf{d} = (d_0, d_1, \ldots, d_{\ell-1})$ where $d_j \in [-2^{b-1}, 2^{b-1})$

$a' \leftarrow a$
mask $\leftarrow 2^b - 1$
for $j = 0$ to $\ell - 1$ **do**
  $u \leftarrow a'$ & mask ▷ Extract the lowest b bits
  carry $\leftarrow u \gg (b-1)$ ▷ Extract the highest bit of u (1 or 0)
  $d_j \leftarrow u - (\text{carry} \ll b)$ ▷ Branchlessly shift to negative range
  $a' \leftarrow (a' \gg b) + \text{carry}$ ▷ Shift down and add the carry
end for
return d

3.2.2 GHS/Hybrid

- Reuses OpenFHE internals, including Hybrid keyswitching settings
- Downside: parameters tied to keyswitching, can be a problem if a program uses both keyswitching and RGSW

Pre-computed Values

For BV:

- $B^i \bmod q_j$ for all i, j
- Get $B = \lceil \max_j \|q_j\| / \hat{\ell} \rceil$ from ℓ

For GHS/Hybrid:

-

3.3 Implementation Details (sPAR)

- Does not implement the bucket sorting step: negligible impact

3.4 Experimental Setup

- Setup phase (considered "free")

Chapter 4
Results and Discussion

- Parameters used and time per user/message of each step in the protocol
- Differences between BV and GHS gadgets for the external product
- AVX512 improvements

Chapter 5
Conclusion and Future Work

Conclusion

– Achieved better time than expected

Future Work

– BGV SIMD capabilities [<**empty citation**>] not utilized

References

[BAB+22] A. A. Badawi, A. Alexandru, *et al.*, *OpenFHE: Open-source fully homomorphic encryption library*, Cryptology ePrint Archive, Paper 2022/915, 2022. [Online]. Available: https://eprint.iacr.org/2022/915.

[BEHZ16] J.-C. Bajard, J. Eynard, *et al.*, *A full RNS variant of FV like somewhat homomorphic encryption schemes*, Cryptology ePrint Archive, Paper 2016/510, 2016. [Online]. Available: https://eprint.iacr.org/2016/510.

[BGV11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, *Fully homomorphic encryption without bootstrapping*, Cryptology ePrint Archive, Paper 2011/277, 2011. [Online]. Available: https://eprint.iacr.org/2011/277.

[BP16] Z. Brakerski and R. Perlman, *Lattice-based fully dynamic multi-key FHE with short ciphertexts*, Cryptology ePrint Archive, Paper 2016/339, 2016. [Online]. Available: https://eprint.iacr.org/2016/339.

[BV11] Z. Brakerski and V. Vaikuntanathan, *Efficient fully homomorphic encryption from (standard) LWE*, Cryptology ePrint Archive, Paper 2011/344, 2011. [Online]. Available: https://eprint.iacr.org/2011/344.

[CGGI16] I. Chillotti, N. Gama, *et al.*, *Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds*, Cryptology ePrint Archive, Paper 2016/870, 2016. [Online]. Available: https://eprint.iacr.org/2016/870.

[CGGI18] I. Chillotti, N. Gama, *et al.*, *TFHE: Fast fully homomorphic encryption over the torus*, Cryptology ePrint Archive, Paper 2018/421, 2018. [Online]. Available: https://eprint.iacr.org/2018/421.

[DM14] L. Ducas and D. Micciancio, *FHEW: Bootstrapping homomorphic encryption in less than a second*, Cryptology ePrint Archive, Paper 2014/816, 2014. [Online]. Available: https://eprint.iacr.org/2014/816.

[Gen09] C. Gentry, "A fully homomorphic encryption scheme", Ph.D. dissertation, Stanford University, 2009. [Online]. Available: https://crypto.stanford.edu/craig.

[GHS12] C. Gentry, S. Halevi, and N. P. Smart, *Homomorphic evaluation of the AES circuit*, Cryptology ePrint Archive, Paper 2012/099, 2012. [Online]. Available: https://eprint.iacr.org/2012/099.

[GSW13] C. Gentry, A. Sahai, and B. Waters, *Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based*, Cryptology ePrint Archive, Paper 2013/340, 2013. [Online]. Available: https://eprint.iacr.org/2013/340.

[HPS18] S. Halevi, Y. Polyakov, and V. Shoup, *An improved RNS variant of the BFV homomorphic encryption scheme*, Cryptology ePrint Archive, Paper 2018/117, 2018. [Online]. Available: https://eprint.iacr.org/2018/117.

[Joy21] M. Joye, *Guide to fully homomorphic encryption over the [discretized] torus*, Cryptology ePrint Archive, Paper 2021/1402, 2021. [Online]. Available: https://eprint.iacr.org/2021/1402.

[KGV14] A. Khedr, G. Gulak, and V. Vaikuntanathan, *SHIELD: Scalable homomorphic implementation of encrypted data-classifiers*, Cryptology ePrint Archive, Paper 2014/838, 2014. [Online]. Available: https://eprint.iacr.org/2014/838.

[KPZ21] A. Kim, Y. Polyakov, and V. Zucca, *Revisiting homomorphic encryption schemes for finite fields*, Cryptology ePrint Archive, Paper 2021/204, 2021. [Online]. Available: https://eprint.iacr.org/2021/204.

[RAD78] R. L. Rivest, L. Adleman, and M. L. Dertouzos, “On data banks and privacy homomorphisms”, in *Foundations of Secure Computation*, 1978, pp. 169–180.

[SEV+15] Y. Sun, A. Edmundson, *et al.*, “RAPTOR: Routing attacks on privacy in tor”, in *24th USENIX Security Symposium (USENIX Security 15)*, Washington, D.C.: USENIX Association, Aug. 2015, pp. 271–286. [Online]. Available: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/sun.

[SW21] E. Shi and K. Wu, *Non-interactive anonymous router*, Cryptology ePrint Archive, Paper 2021/435, 2021. [Online]. Available: https://eprint.iacr.org/2021/435.

Appendix A
Number Theory

A.1 Chinese Remaineder Theorem

Could be stated here just to be more complete

A.2 Number Theoretic Transform

- Turns a convolution into a nega/cyclic
- Same as FFT but over different ring